

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID Principle



Disusun Oleh:

Erlangga Aditya Permana 105224038

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

SOLID Principle merupakan singkatan dari lima prinsip desain berorientasi objek yang diperkenalkan oleh Robert C. Martin. Kelima prinsip tersebut meliputi *Single Responsibility Principle* (SRP), *Open/Closed Principle* (OCP), *Liskov Substitution Principle* (LSP), *Interface Segregation Principle* (ISP), dan *Dependency Inversion Principle* (DIP). Tujuan utama dari penerapan prinsip SOLID adalah untuk menciptakan struktur kode perangkat lunak yang terkelola dengan baik, mudah dikembangkan, tahan terhadap perubahan, serta meminimalisir dampak sistemik ketika terdapat penambahan fitur atau modifikasi logika bisnis di masa depan.

Program yang dirancang dalam laporan ini adalah bentuk rancang ulang terhadap sistem pengisian Kartu Rencana Studi (KRS) pada portal akademik (SIAKAD) yang sebelumnya kaku. Program ini dirombak total dengan memecah satu kelas yang mengatur segala hal menjadi kelas-kelas terpisah dengan tanggung jawab tunggal. Selain itu, program menangani kalkulasi UKT yang fleksibel, memisahkan *interface* fungsional spesifik untuk berbagai jenis kelas, serta menggunakan abstraksi agar sistem bisa menerima injeksi jenis database apa pun. Melalui perbaikan arsitektur ini, sistem aplikasi dipastikan kokoh dan siap memfasilitasi kebutuhan migrasi server serta aturan akademik baru tanpa mengganggu operasional sistem yang sudah berjalan dengan stabil.

II. Variable

No	Nama Variabel	Tipe Data	Fungsi
1	skemaKalkulasi	SkemaKalkulasiUKT	Menyimpan referensi skema perhitungan UKT (Reguler, MBKM, dll) sesuai prinsip abstraksi pada OCP.
2	storage	DatabaseStorage	Menyimpan referensi jenis media/database (MySQL/NoSQL) tujuan untuk pengiriman data (DIP).
3	validator	KRSValidator	Objek delegasi untuk secara khusus menangani logika validasi prasyarat KRS (SRP).
4	pdfGenerator	KRSPdfGenerator	Objek delegasi untuk membangun kerangka dan mencetak laporan KRS ke bentuk <i>softcopy</i> PDF (SRP).

No	Nama Variabel	Tipe Data	Fungsi
5	repository	KRSRepository	Objek delegasi untuk melayani logika transaksi persistensi rekaman pengambilan mata kuliah (SRP).

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	UKTCalculator()	Constructor	Menginisialisasi kalkulator biaya beserta injeksi spesifik skema UKT yang sedang beroperasi.
2	KRSRepository()	Constructor	Membangun komponen repositori dengan menanamkan dependensi platform <i>database</i> yang digunakan.
3	SistemKRSTManager()	Constructor	Menginisialisasi komponen inti manajemen KRS dan menjembatani <i>injection</i> untuk arsitektur <i>database</i> .
4	validasiPrasyarat()	Void	Memeriksa apakah mahasiswa memenuhi prasyarat untuk mata kuliah yang akan diambil.
5	cetakDrafPDF()	Void	Mengeksekusi penulisan grafis dan teks draf KRS ke dalam bentuk cetakan <i>file</i> PDF.
6	hitungUKT()	Void	Menjalankan formula matematika perhitungan biaya pendidikan sesuai jalur masuk mahasiswa.

No	Nama Metode	Jenis Metode	Fungsi
7	jalankanKalkulasi() ()	Void	Memicu kalkulator utama agar menjalankan implementasi spesifik dari skema UKT yang telah ditugaskan.
8	infoMataKuliah()	Void	Mengambil serta menampilkan basis data dan spesifikasi fundamental untuk jenis mata kuliah.
9	alokasiAsistenLab() ()	Void	Khusus mengatur penugasan entitas asisten dan waktu ke laboratorium untuk pelaksanaan praktikum.
10	cekPeralatanPraktikum() ()	Void	Memeriksa fungsi dan inventaris perangkat keras pada laboratorium pra-dimulainya praktikum.
11	simpanDataKRS()	Void	Mengeksekusi penyisipan kueri log riwayat KRS langsung ke <i>backend</i> (MySQL/NoSQL).
12	simpanRiwayat()	Void	Mengelola dan menjembatani perintah tingkat tinggi untuk sinkronisasi ke objek persistensi data.
13	prosesPengisianKRS() ()	Void	Mengatur sekuens pendaftaran dari validasi, bayar, hingga penyimpanan.

IV. Dokumentasi dan Pembahasan Code

a. Isi Kode

DatabaseStorage.java :

```
public interface DatabaseStorage {
```

```
void simpanDataKRS();  
}
```

KalkulasiKaryawan.java :

```
public class KalkulasiKaryawan implements SkemaKalkulasiUKT {  
  
    @Override  
  
    public void hitungUKT() {  
  
        System.out.println("Menghitung UKT untuk jalur Karyawan...");  
  
    }  
  
}
```

KalkulasiMBKM.java :

```
public class KalkulasiMBKM implements SkemaKalkulasiUKT {  
  
    @Override  
  
    public void hitungUKT() {  
  
        System.out.println("Menghitung UKT/SKS khusus untuk program MBKM...");  
  
    }  
  
}
```

KalkulasiReguler.java :

```
public class KalkulasiReguler implements SkemaKalkulasiUKT {  
  
    @Override  
  
    public void hitungUKT() {  
  
        System.out.println("Menghitung UKT untuk jalur Reguler...");  
  
    }  
  
}
```

KRSPdfGenerator.java :

```
public class KRSPdfGenerator {  
  
    public void cetakDrafPDF() {  
  
        System.out.println("Mencetak draf KRS ke dalam bentuk file PDF...");  
  
    }  
  
}
```

KRSRepository.java :

```
public class KRSRepository {  
  
    private DatabaseStorage storage;  
  
    public KRSRepository(DatabaseStorage storage) {  
  
        this.storage = storage;  
  
    }  
  
    public void simpanRiwayat() {  
  
        storage.simpanDataKRS();  
  
    }  
  
}
```

KRSValidator.java :

```
public class KRSValidator {  
  
    public void validasiPrasyarat() {  
  
        System.out.println("Sedang memvalidasi prasyarat mata kuliah mahasiswa...");  
  
    }  
  
}
```

```
}  
  
}
```

MataKuliahDasar.java :

```
public interface MataKuliahDasar {  
  
    void infoMataKuliah();  
  
}
```

MataKuliahKKN.java :

```
public class MataKuliahKKN implements MataKuliahDasar {  
  
    @Override  
  
    public void infoMataKuliah() {  
  
        System.out.println("Memproses mata kuliah KKN (di lapangan  
lepas)...");  
  
    }  
  
}
```

MataKuliahPraktikum.java :

```
public class MataKuliahPraktikum implements MataKuliahDasar, OperasiPraktikum  
{  
  
    @Override  
  
    public void infoMataKuliah() {  
  
        System.out.println("Memproses mata kuliah Praktikum...");  
  
    }  
  
    @Override  
  
    public void alokasiAsistenLab() {
```



```

        System.out.println("Sedang mengalokasikan asisten lab...");

    }

    @Override

    public void cekPeralatanPraktikum() {

        System.out.println("Sedang mengecek peralatan praktikum...");

    }

}

```

MataKuliahTeori.java :

```

public class MataKuliahTeori implements MataKuliahDasar {

    @Override

    public void infoMataKuliah() {

        System.out.println("Memproses mata kuliah Teori (tidak butuh asisten lab)...");

    }

}

```

MySQLDatabaseConnection.java :

```

public class MySQLDatabaseConnection implements DatabaseStorage {

    @Override

    public void simpanDataKRS() {

        System.out.println("Menyimpan data riwayat KRS ke database lama MySQL...");

    }

}

```

NoSQLCloudConnection.java :

```
public class NoSQLCloudConnection implements DatabaseStorage {

    @Override

    public void simpanDataKRS() {

        System.out.println("Menyimpan data riwayat KRS ke infrastruktur Cloud  
NoSQL terbaru...");

    }

}
```

OperasiPraktikum.java :

```
public interface OperasiPraktikum {

    void alokasiAsistenLab();

    void cekPeralatanPraktikum();

}
```

SistemKRSManger.java :

```
public class SistemKRSManger {

    private KRSValidator validator;

    private KRSPdfGenerator pdfGenerator;

    private KRSRepository repository;

    public SistemKRSManger(DatabaseStorage storage) {

        this.validator = new KRSValidator();

        this.pdfGenerator = new KRSPdfGenerator();

        this.repository = new KRSRepository(storage);

    }

}
```

```

    }

    public void prosesPengisianKRS(SkemaKalkulasiUKT skemaUKT) {

        System.out.println("--- Memulai Proses KRS ---");

        validator.validasiPrasyarat();

        UKTCalculator calculator = new UKTCalculator(skemaUKT);

        calculator.jalankanKalkulasi();

        pdfGenerator.cetakDrafPDF();

        repository.simpanRiwayat();

        System.out.println("--- Proses KRS Selesai ---\n");

    }
}

```

SkemaKalkulasiUKT.java :

```

public interface SkemaKalkulasiUKT {

    void hitungUKT();

}

```

UKTCalculator :

```

public class UKTCalculator {

    private SkemaKalkulasiUKT skemaKalkulasi;
}

```

```

public UKTCalculator(SkemaKalkulasiUKT skemaKalkulasi) {

    this.skemaKalkulasi = skemaKalkulasi;

}

public void jalankanKalkulasi() {

    skemaKalkulasi.hitungUKT();

}

}

```

Main.java :

```

public class Main {

    public static void main(String[] args) {

        DatabaseStorage databaseMasaDepan = new NoSQLCloudConnection();

        SistemKRSManger krsManager = new SistemKRSManger(databaseMasaDepan);

        SkemaKalkulasiUKT mahasiswaMBKM = new KalkulasiMBKM();

        krsManager.prosesPengisianKRS(mahasiswaMBKM);

        System.out.println("--- Simulasi Mata Kuliah ---");

        MataKuliahDasar mkTeori = new MataKuliahTeori();

        mkTeori.infoMataKuliah();
    }
}

```

```

MataKuliahPraktikum mkPraktikum = new MataKuliahPraktikum();

mkPraktikum.infoMataKuliah();

mkPraktikum.cekPeralatanPraktikum();

}
}

```

b. Alur Program

1. Inisialisasi Dependensi Database

Program diawali dengan mendefinisikan infrastruktur penyimpanan secara abstrak. Objek dari platform database konkrit (misalnya NoSQLCloudConnection) dipersiapkan terlebih dahulu dan disuntikkan ke dalam SistemKRSManager. Langkah ini membebaskan logika bisnis pusat dari rantai teknologi yang spesifik, mempermudah rencana Rektorat dalam bermigrasi data.

2. Penentuan Strategi Logika UKT

Sistem mendeteksi jalur profil mahasiswa dan membuat objek abstraksi SkemaKalkulasiUKT yang relevan. Jika mahasiswa mengambil jalur MBKM, sistem cukup melampirkan modul fungsi KalkulasiMBKM ke dalam aliran operasi.

3. Orkestrasi Pendaftaran KRS Dimulai

Proses utama dimulai ketika pemanggilan prosesPengisianKRS() dieksekusi. Tahapan awal secara otomatis diserahkan pada fungsionalitas KRSValidator untuk memastikan bahwa kelayakan akademik mahasiswa terhadap subjek mata kuliah terpenuhi secara mutlak.

4. Kalkulasi Tagihan Secara Otomatis

Tanpa menggunakan evaluasi *If-Else* yang panjang, manajer mendelegasikan tugas penghitungan nominal finansial pada kelas UKTCalculator.

5. Percetakan Laporan Administrasi Dokumen

Setelah status akademik dinilai valid dan beban finansial sukses diurai, tahapan dialihkan kepada layanan spesifik KRSPdfGenerator guna merender lembaran resmi tanda terima pendaftaran akademik (KRS) berformat PDF mahasiswa yang bersangkutan.

6. Persistensi Log Aktivitas Mahasiswa

Sebagai langkah akhir orkestrasi internal, SistemKRSManager mengarahkan KRSRepository untuk melakukan *commit* terhadap catatan operasional. Repository tersebut langsung mengirimnya melalui *interface* kepada arsitektur *Cloud NoSQL* sesuai dengan alur di titik awal.

7. Iterasi Spesifik Objek Mata Kuliah

Sistem berjalan stabil mensimulasikan data iterasi kelas. Jika memanipulasi kelas *Teori* maupun *KKN*, program hanya mengambil info mendasar mereka. Namun untuk objek kelas *Praktikum*, program mengeksekusi operasi pentingnya, yaitu persiapan inventaris dan asisten laboratorium tanpa khawatir merusak rantai objek lain karena antarmuka yang terisolasi aman.

c. Penerapan Prinsip SOLID dalam Alur Program

Secara keseluruhan, alur eksekusi di atas mencerminkan perbaikan arsitektur berlandaskan kelima prinsip SOLID. Pemenuhan prinsip Single Responsibility (SRP) terlihat nyata dengan didelegasikannya beban raksasa dari SistemKRSManager ke dalam kelas-kelas independen seperti KRSValidator, KRSPdfGenerator, dan KRSRepository. Untuk membebaskan sistem dari kode logika kondisional yang berisiko, prinsip Open/Closed (OCP) diterapkan lewat penciptaan *interface* SkemaKalkulasiUKT beserta sub-implementasinya seperti KalkulasiMBKM dan KalkulasiReguler. Pada area objek studi, pencegahan error *exception* berhasil diwujudkan melalui Interface Segregation (ISP) dan Liskov Substitution (LSP), antarmuka besar dipecah menjadi MataKuliahDasar dan OperasiPraktikum, sehingga MataKuliahTeori dan MataKuliahKKN terbebas dari paksaan memuat fungsionalitas laboratorium yang kini murni menjadi hak adopsi MataKuliahPraktikum. Pada struktur terdalamnya, integrasi migrasi data dimungkinkan berkat Dependency Inversion (DIP), di mana modul manajer inti maupun KRSRepository dipaksa patuh untuk hanya bergantung pada abstraksi DatabaseStorage, alih-alih terikat secara statis pada rupa fisik MySQLDatabaseConnection ataupun NoSQLCloudConnection.

V. Kesimpulan

Tindakan perancangan ulang yang dieksekusi pada basis kode internal sistem portal akademik memvalidasi kekuatan dari prinsip *SOLID* dalam mengurangi kecacatan desain program. Dengan membongkar dominasi kelas tunggal menjadi kelas-kelas terpisah yang memiliki tanggung jawab masing-masing (SRP), error trivial pada manipulasi PDF kini mustahil menyebabkan kerusakan database lagi. Transisi menuju pola struktural berbasis implementasi spesifik (OCP) menghilangkan kerumitan ekspresi iterasi berlebih, memberikan Rektorat otoritas tak terbatas dalam memperkenalkan jalur beasiswa baru atau program khusus layaknya MBKM.

Resolusi di antarmuka kelas telah yang memaksa kelas untuk melempar respon *exception* (pemenuhan ISP dan LSP), memuluskan *looping* iterasi di saat jam padat (seperti skenario War KRS). Kemenangan arsitektur terbesar tercapai saat lapisan aplikasi tingkat tinggi dihilangkan ketergantungannya dari basis data MySQL yang *dependent* (DIP). Restrukturisasi ini bukan hanya sekadar mengembalikan fungsi aplikasi, namun mengubahnya menjadi sebuah utilitas *software enterprise* yang kokoh secara fundamental dan responsif terhadap dorongan inovasi di masa depan.